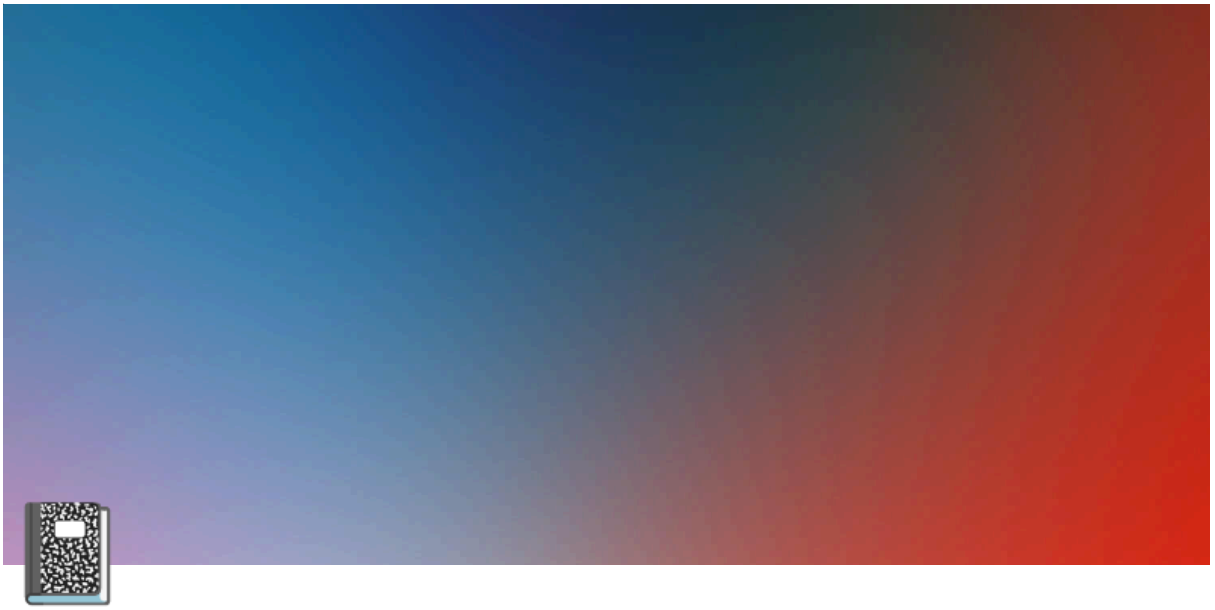

Miniprojet 1 — Pipeline ETL & Data Warehouse

Mexora Analytics — Construire l'entrepôt de données from scratch

> **NB** : à rendre avant le 1er Juin 2026,
> **Livrable** : *Rapport PDF (incluant repository Github),*
> **Plateforme** : ClassRoom (*ne pas envoyer vos travaux par email*),
> **Important** : La date de livraison est cruciale et est comptée dans l'évaluation du travail !

📢 Indication

> Les scripts Python et SQL fournis dans cet énoncé sont donnés **à titre indicatif**. Vous devez vous assurer de leur bon fonctionnement avant de les utiliser.
> Les fichiers de données sont **à générer par vos soins**, en respectant les structures et les problèmes intentionnels décrits dans chaque section. Aucun fichier ne sera fourni.
> Vous êtes **libres** de formuler vos propres hypothèses et choix de conception, à condition qu'ils soient cohérents avec l'énoncé et explicitement justifiés

> dans votre rapport. Un choix assumé et argumenté sera toujours valorisé, même s'il diffère de la solution attendue.

🎯 Contexte et mise en situation

Vous venez d'être recruté(e) comme **Data Engineer junior** chez **Mexora**, une marketplace e-commerce basée à Tanger. L'entreprise vend des produits électroniques, de mode et d'alimentation à des clients dans tout le Maroc.

Mexora gère ses opérations quotidiennes via un système transactionnel MySQL. Chaque commande, chaque paiement, chaque livraison est enregistré en temps réel. Mais le Directeur Général vient de formuler une demande urgente :

> *"Je veux pouvoir répondre à des questions comme : quels sont nos meilleurs clients à Tanger ce trimestre ? Quelle catégorie de produit performe le mieux pendant le Ramadan ? Quel est notre taux de retour par région ? Et je veux ces réponses en quelques secondes, pas en 6 minutes."*

Votre mission : **concevoir et implémenter un système décisionnel complet**, depuis la modélisation jusqu'au dashboard final.

📋 Objectifs pédagogiques

À la fin de ce miniprojet, vous serez capable de :

- Concevoir un **modèle dimensionnel** (schéma en étoile) adapté à un cas réel
- Identifier et justifier la **granularité** d'une table de faits
- Gérer les **Slowly Changing Dimensions (SCD)** de type 1 et 2
- Développer un **pipeline ETL complet** en Python (extraction, transformation, chargement)
- Implémenter un **Data Warehouse** sous PostgreSQL avec indexation et vues matérialisées
- Créer un **dashboard décisionnel** répondant à des questions métier précises
- Documenter chaque choix architectural de manière professionnelle

---

📦 Données fournies

Vous disposez de **4 fichiers de données brutes** intentionnellement imparfaits — comme dans la réalité.

Fichier 1 — `commandes_mexora.csv` (50 000 lignes)

```
id_commande, id_client, id_produit, date_commande, quantite, prix_unitaire,
statut, ville_livraison, mode_paiement, id_livreur, date_livraison
```

Problèmes intentionnels à détecter et corriger :
- Doublons sur `id_commande` (environ 3%)
- Dates en formats mixtes : `15/11/2024`, `2024-11-15`, `Nov 15 2024`
- `ville_livraison` incohérente : `"tanger"`, `"TNG"`, `"TANGER"`, `"Tnja"`
- Valeurs manquantes sur `id_livreur` (7% des lignes)
- `quantite` négative sur quelques lignes (erreurs de saisie)
- `prix_unitaire` à 0 sur certaines lignes (commandes test à supprimer)
- `statut` avec des valeurs non standards : `"OK"`, `"KO"`, `"DONE"`

Fichier 2 — `produits_mexora.json`

```
{
  "produits": [
    {
      "id_produit": "P001",
      "nom": "iPhone 16 Pro 256Go",
      "categorie": "Electronique",
      "sous_categorie": "Smartphones",
      "marque": "Apple",
      "fournisseur": "Apple MENA",
      "prix_catalogue": 12999.00,
      "origine_pays": "USA",
      "date_creation": "2024-09-20",
      "actif": true
    },
    ...
  ]
}
```

Problèmes intentionnels :
- Catégories avec casse incohérente : `"electronique"`, `"Electronique"`, `"ELECTRONIQUE"`
- Certains produits ont `"actif": false` mais ont des commandes associées (à gérer en SCD)
- Prix catalogue à `null` pour quelques produits anciens

Fichier 3 — `clients_mexora.csv`

```
id_client, nom, prenom, email, date_naissance, sexe,
ville, telephone, date_inscription, canal_acquisition
```

Problèmes intentionnels :
- Clients en doublon (même email, `id_client` différent — erreur de migration)
- `sexe` codé différemment selon la source d'import : `m/f`, `1/0`, `Homme/Femme`
- `date_naissance` permettant de calculer des âges négatifs ou > 120 ans (à invalider)
- `ville` avec les mêmes incohérences géographiques que les commandes
- Emails mal formatés (sans `@`, sans domaine)

Fichier 4 — `regions_maroc.csv`

```
code_ville, nom_ville_standard, province, region_admin,
zone_geo, population, code_postal
```

Ce fichier est le **référentiel géographique officiel** — propre et complet. Il sert de table de correspondance pour harmoniser toutes les villes des autres fichiers.

🏗️ Travail demandé

Le projet se décompose en **4 étapes évaluées indépendamment**.

Étape 1 — Modélisation du Data Warehouse (25%)

1.1 Analyse des besoins décisionnels

Avant de modéliser, vous devez formaliser les besoins analytiques de Mexora sous forme de **requêtes-types** (méthode Top-Down vue en cours).

Pour chaque requête-type, utilisez la structure en 3 clauses :

```
Requête R1 :
  Analyser   → le chiffre d'affaires (montant TTC)
  En fonction de → la région, la catégorie de produit, le trimestre
  Pour → les commandes avec statut "livré" uniquement

Requête R2 :
  Analyser   → la quantité vendue et le montant TTC
  En fonction de → le produit, le mois, le segment client
  Pour → l'année 2024

[Vous devez formuler au minimum 5 requêtes-types à partir des questions métier ci-dess
ous]
```

Questions métier à couvrir obligatoirement :
1. Quelle région génère le plus de CA ? Évolution sur 12 mois ?
2. Quels sont les 10 produits les plus vendus par trimestre à Tanger ?
3. Quel segment client a le panier moyen le plus élevé ?
4. Quel est le taux de retour par catégorie de produit ?
5. Y a-t-il un effet Ramadan visible sur les ventes d'alimentation ?

1.2 Conception du schéma en étoile

Livrable attendu : **schéma entité-relation complet** (outil libre : draw.io, Lucidchart, dbdiagram.io) avec :

Table de faits FAIT_VENTES :
- Définir toutes les clés étrangères
- Lister toutes les mesures avec leur **type d'additivité** (additive / semi-additive / non-additive)
- Justifier la granularité choisie : qu'est-ce qu'une ligne représente exactement ?

Tables de dimensions :
Concevoir au minimum les 5 dimensions suivantes avec tous leurs attributs :

Dimension	Attributs minimum requis
`DIM_TEMPS`	id_date, jour, mois, trimestre, annee, libelle_mois, est_weekend, est_ferie_maroc, periode_ramadan
`DIM_PRODUIT`	id_produit, nom_produit, categorie, sous_categorie, marque, fournisseur, prix_standard, origine_pays

Dimension	Attributs minimum requis
`DIM_CLIENT`	id_client, nom_complet, tranche_age, sexe, ville, region, segment_client, canal_acquisition
`DIM_REGION`	id_region, ville, province, region_admin, zone_geo
`DIM_LIVREUR`	id_livreur, nom_livreur, type_transport, zone_couverture

1.3 Gestion des SCD

Identifier dans vos données au moins **2 cas de Slowly Changing Dimensions** et justifier le type choisi (SCD 1, 2 ou 3) pour chacun.

Exemple attendu :
> "Le produit iPhone 16 passe de la catégorie 'Téléphones' à 'Smartphones' en mars 2024. Nous choisissons SCD Type 2 car nous voulons que les ventes antérieures à mars restent classées dans 'Téléphones' pour la cohérence historique."

Livrable Étape 1

- Schéma entité-relation annoté (image ou lien)
- Document de 2 pages maximum justifiant :
  - La granularité choisie
  - Les mesures et leur additivité
  - Les 2 cas de SCD avec le type retenu et sa justification

Étape 2 — Pipeline ETL en Python (40%)

2.1 Structure du projet Python

Votre code doit être organisé de façon professionnelle :

```
mexora_etl/
├── config/
│   └── settings.py          # paramètres de connexion, chemins
├── extract/
│   └── extractor.py         # fonctions d'extraction par source
├── transform/
│   ├── clean_commandes.py   # nettoyage des commandes
│   ├── clean_clients.py     # nettoyage des clients
│   ├── clean_produits.py    # nettoyage des produits
│   └── build_dimensions.py  # construction des dimensions
├── load/
```

```
│   └── loader.py           # chargement dans PostgreSQL
├── utils/
│   └── logger.py           # logging des opérations ETL
├── main.py                 # orchestration du pipeline
└── requirements.txt
```

2.2 Phase Extract

Extraction des commandes (CSV) :

```
import pandas as pd
import logging

def extract_commandes(filepath: str) -> pd.DataFrame:
    """
    Extrait les commandes depuis le fichier CSV source.
    Retourne le DataFrame brut sans modification.
    """
    df = pd.read_csv(filepath, encoding='utf-8', dtype=str)
# tout en str pour éviter les conversions implicites
    logging.info(f"[EXTRACT] Commandes :{len(df)} lignes ex
traites depuis{filepath}")
    return df

def extract_produits(filepath: str) -> pd.DataFrame:
    """Extrait les produits depuis le fichier JSON."""
    import json
    with open(filepath, 'r', encoding='utf-8') as f:
        data = json.load(f)
    df = pd.DataFrame(data['produits'])
    logging.info(f"[EXTRACT] Produits :{len(df)} lignes ext
raites")
    return df
```

2.3 Phase Transform

Pour chaque transformation, vous devez :

1. Écrire le code de transformation
2. **Logger** le nombre de lignes avant et après
3. **Documenter** la règle métier appliquée en commentaire

Transformations obligatoires sur les commandes :

```
def transform_commandes(df: pd.DataFrame) -> pd.DataFrame:
    """
    Applique l'ensemble des règles de nettoyage sur les com
mandes Mexora.

    Règles appliquées :
      R1 - Suppression des doublons sur id_commande (conser
ver la dernière occurrence)
      R2 - Standardisation des dates (format cible : YYYY-M
M-DD)
      R3 - Harmonisation des noms de villes via le référent
iel régions_maroc
      R4 - Standardisation des statuts de commande
      R5 - Suppression des lignes avec quantite <= 0
      R6 - Suppression des lignes avec prix_unitaire = 0 (c
ommandes test)
      R7 - Remplacement des id_livreur manquants par la val
eur -1 (livreur inconnu)
    """
    initial = len(df)

    # R1 — Suppression des doublons
    df = df.drop_duplicates(subset=['id_commande'], keep='l
ast')
    logging.info(f"[TRANSFORM] R1 doublons :{initial - len
(df)} lignes supprimées")

    # R2 — Standardisation des dates
    df['date_commande'] = pd.to_datetime(
        df['date_commande'], format='mixed', dayfirst=True,
errors='coerce'
    )
    dates_invalides = df['date_commande'].isna().sum()
    df = df.dropna(subset=['date_commande'])
    logging.info(f"[TRANSFORM] R2 dates :{dates_invalides}
dates invalides supprimées")
```

```
    # R3 — Harmonisation des villes
    mapping_villes = charger_referentiel_villes('data/regio
ns_maroc.csv')
    df['ville_livraison'] = df['ville_livraison'].str.strip
().str.lower()
    df['ville_livraison'] = df['ville_livraison'].map(mappi
ng_villes).fillna('Non renseignée')

    # R4 — Standardisation des statuts
    mapping_statuts = {
        'livré': 'livré', 'livre': 'livré', 'LIVRE': 'livr
é', 'DONE': 'livré',
        'annulé': 'annulé', 'annule': 'annulé', 'KO': 'annu
lé',
        'en_cours': 'en_cours', 'OK': 'en_cours',
        'retourné': 'retourné', 'retourne': 'retourné'
    }
    df['statut'] = df['statut'].replace(mapping_statuts)
    invalides = ~df['statut'].isin(['livré', 'annulé', 'en_
cours', 'retourné'])
    logging.warning(f"[TRANSFORM] R4 statuts :{invalides.su
m()} valeurs non reconnues → 'inconnu'")
    df.loc[invalides, 'statut'] = 'inconnu'

    # R5 — Quantités invalides
    avant = len(df)
    df = df[df['quantite'].astype(float) > 0]
    logging.info(f"[TRANSFORM] R5 quantités :{avant - len(d
f)} lignes supprimées (quantité <= 0)")

    # R6 — Prix nuls (commandes test)
    avant = len(df)
    df = df[df['prix_unitaire'].astype(float) > 0]
    logging.info(f"[TRANSFORM] R6 prix :{avant - len(df)} c
ommandes test supprimées")

    # R7 — Livreurs manquants
```

```
    nb_manquants = df['id_livreur'].isna().sum()
    df['id_livreur'] = df['id_livreur'].fillna('-1')
    logging.info(f"[TRANSFORM] R7 livreurs :{nb_manquants}
valeurs manquantes remplacées par -1")

    logging.info(f"[TRANSFORM] Commandes :{initial} →{len(d
f)} lignes ({initial - len(df)} supprimées au total)")
    return df
```

Transformations obligatoires sur les clients :

```
def transform_clients(df: pd.DataFrame) -> pd.DataFrame:
    """
    Règles :
      R1 - Déduplication sur email normalisé (conserver ins
cription la plus récente)
      R2 - Standardisation du sexe (cible : 'm' / 'f' / 'in
connu')
      R3 - Validation des dates de naissance (âge entre 16
et 100 ans)
      R4 - Validation du format email
      R5 - Segmentation client Gold/Silver/Bronze (calculée
depuis les commandes)
    """
    # R1 — Déduplication
    df['email_norm'] = df['email'].str.lower().str.strip()
    df = df.sort_values('date_inscription').drop_duplicates
(subset=['email_norm'], keep='last')

    # R2 — Standardisation du sexe
    mapping_sexe = {
        'm': 'm', 'f': 'f', '1': 'm', '0': 'f',
        'homme': 'm', 'femme': 'f', 'male': 'm', 'female':
'f',
        'h': 'm'
    }
    df['sexe'] = df['sexe'].str.lower().str.strip().map(map
ping_sexe).fillna('inconnu')
```

```
    # R3 — Validation des dates de naissance
    from datetime import date
    df['date_naissance'] = pd.to_datetime(df['date_naissanc
e'], errors='coerce')
    today = pd.Timestamp(date.today())
    df['age'] = (today - df['date_naissance']).dt.days // 3
65
    df.loc[(df['age'] < 16) | (df['age'] > 100), 'date_nais
sance'] = pd.NaT
    df['tranche_age'] = pd.cut(
        df['age'].fillna(0),
        bins=[0, 18, 25, 35, 45, 55, 65, 200],
        labels=['<18', '18-24', '25-34', '35-44', '45-54',
'55-64', '65+']
    )

    # R4 — Validation email
    import re
    pattern_email = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a
-zA-Z]{2,}$'
    df.loc[~df['email'].str.match(pattern_email, na=False),
'email'] = None

    return df
```

Construction de la dimension temporelle :

```
def build_dim_temps(date_debut: str, date_fin: str) -> pd.D
ataFrame:
    """
    Génère la dimension temporelle complète entre deux date
s.
    Inclut les jours fériés marocains et les périodes Ramad
an.
    """
    dates = pd.date_range(start=date_debut, end=date_fin, f
req='D')
```

```
    # Jours fériés marocains (liste partielle — à compléte
r)
    feries_maroc = [
        '2024-01-01',  # Nouvel An
        '2024-01-11',  # Manifeste de l'Indépendance
        '2024-05-01',  # Fête du Travail
        '2024-07-30',  # Fête du Trône
        '2024-08-14',  # Allégeance Oued Eddahab
        '2024-11-06',  # Marche Verte
        '2024-11-18',  # Fête de l'Indépendance
    ]

    # Périodes Ramadan 2022-2025 (approximatif)
    ramadan_periodes = [
        ('2022-04-02', '2022-05-01'),
        ('2023-03-22', '2023-04-20'),
        ('2024-03-10', '2024-04-09'),
    ]

    df = pd.DataFrame({
        'id_date':       dates.strftime('%Y%m%d').astype(in
t),
        'date_complete': dates,
        'jour':          dates.day,
        'mois':          dates.month,
        'trimestre':     dates.quarter,
        'annee':         dates.year,
        'semaine':       dates.isocalendar().week,
        'libelle_jour':  dates.strftime('%A'),
        'libelle_mois':  dates.strftime('%B'),
        'est_weekend':   dates.dayofweek >= 5,
        'est_ferie_maroc': dates.strftime('%Y-%m-%d').isin
(feries_maroc),
    })

    # Calcul période Ramadan
    df['periode_ramadan'] = False
```

```
    for debut, fin in ramadan_periodes:
        masque = (df['date_complete'] >= debut) & (df['date
_complete'] <= fin)
        df.loc[masque, 'periode_ramadan'] = True

    return df[['id_date', 'jour', 'mois', 'trimestre', 'ann
ee', 'semaine',
               'libelle_jour', 'libelle_mois', 'est_weeken
d',
               'est_ferie_maroc', 'periode_ramadan']]
```

Calcul de la segmentation client (mesure dérivée) :

```
def calculer_segments_clients(df_commandes: pd.DataFrame) -
> pd.DataFrame:
    """
    Calcule le segment client (Gold/Silver/Bronze) basé sur
le CA cumulé
    des 12 derniers mois pour chaque client.

    Règles métier Mexora :
      Gold   : CA 12 mois >= 15 000 MAD
      Silver : CA 12 mois >= 5 000 MAD
      Bronze : CA 12 mois < 5 000 MAD
    """
    from datetime import date, timedelta

    date_limite = pd.Timestamp(date.today() - timedelta(day
s=365))

    df_recents = df_commandes[
        (df_commandes['date_commande'] >= date_limite) &
        (df_commandes['statut'] == 'livré')
    ].copy()

    df_recents['montant_ttc'] = df_recents['quantite'].asty
pe(float) * df_recents['prix_unitaire'].astype(float)
```

```
    ca_par_client = df_recents.groupby('id_client')['montan
t_ttc'].sum().reset_index()
    ca_par_client.columns = ['id_client', 'ca_12m']

    def segmenter(ca):
        if ca >= 15000: return 'Gold'
        elif ca >= 5000: return 'Silver'
        else: return 'Bronze'

    ca_par_client['segment_client'] = ca_par_client['ca_12
m'].apply(segmenter)
    return ca_par_client[['id_client', 'segment_client', 'c
a_12m']]
```

2.4 Phase Load

```
import sqlalchemy
import pandas as pd

def charger_dimension(df: pd.DataFrame, table_name: str, en
gine, if_exists='replace'):
    """
    Charge une table de dimension dans PostgreSQL.
    Stratégie : replace (truncate + reload) pour les dimens
ions.
    """
    df.to_sql(
        name=table_name,
        con=engine,
        schema='dwh_mexora',
        if_exists=if_exists,
        index=False,
        method='multi',
        chunksize=1000
    )
    print(f"[LOAD]{table_name} :{len(df)} lignes chargées")

def charger_faits(df: pd.DataFrame, engine):
```

```
    """
    Charge la table de faits avec une stratégie UPSERT.
    Utilise ON CONFLICT pour mettre à jour les lignes exist
antes.
    """
    from sqlalchemy.dialects.postgresql import insert

    # Construction de l'upsert via SQLAlchemy core
    with engine.connect() as conn:
        for chunk in [df[i:i+5000] for i in range(0, len(d
f), 5000)]:
            records = chunk.to_dict('records')
            stmt = insert(table_fait_ventes).values(record
s)
            stmt = stmt.on_conflict_do_update(
                index_elements=['id_vente'],
                set_={c.key: c for c in stmt.excluded if c.
key != 'id_vente'}
            )
            conn.execute(stmt)
            conn.commit()
    print(f"[LOAD] fait_ventes :{len(df)} lignes chargées
(upsert)")
```

2.5 Orchestration du pipeline

```
# main.py — point d'entrée du pipeline ETL Mexora

import logging
from datetime import datetime

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s —%(levelname)s —%(message)s',
    handlers=[
        logging.FileHandler(f'logs/etl_{datetime.now().strf
time("%Y%m%d_%H%M%S")}.log'),
        logging.StreamHandler()
```

```
    ]
)

def run_pipeline():
    start = datetime.now()
    logging.info("=" * 60)
    logging.info("DÉMARRAGE PIPELINE ETL MEXORA")
    logging.info("=" * 60)

    try:
        # 1. EXTRACT
        logging.info("--- PHASE EXTRACT ---")
        df_commandes_raw = extract_commandes('data/commande
s_mexora.csv')
        df_produits_raw  = extract_produits('data/produits_
mexora.json')
        df_clients_raw   = extract_clients('data/clients_me
xora.csv')
        df_regions       = extract_regions('data/regions_ma
roc.csv')

        # 2. TRANSFORM
        logging.info("--- PHASE TRANSFORM ---")
        df_commandes = transform_commandes(df_commandes_ra
w)
        df_clients   = transform_clients(df_clients_raw)
        df_produits  = transform_produits(df_produits_raw)

        dim_temps    = build_dim_temps('2020-01-01', '2025-
12-31')
        dim_client   = build_dim_client(df_clients, df_comm
andes)
        dim_produit  = build_dim_produit(df_produits)
        dim_region   = build_dim_region(df_regions)
        dim_livreur  = build_dim_livreur(df_commandes)
        fait_ventes  = build_fait_ventes(df_commandes, dim_
temps, dim_client,
                                          dim_produit, dim_
```

```
region, dim_livreur)

        # 3. LOAD
        logging.info("--- PHASE LOAD ---")
        engine = sqlalchemy.create_engine("postgresql://use
r:pass@localhost/mexora_dwh")

        charger_dimension(dim_temps,   'dim_temps',   engin
e)
        charger_dimension(dim_client,  'dim_client',  engin
e)
        charger_dimension(dim_produit, 'dim_produit', engin
e)
        charger_dimension(dim_region,  'dim_region',  engin
e)
        charger_dimension(dim_livreur, 'dim_livreur', engin
e)
        charger_faits(fait_ventes, engine)

        duree = (datetime.now() - start).seconds
        logging.info(f"PIPELINE TERMINÉ EN{duree} seconde
s")

    except Exception as e:
        logging.error(f"ERREUR PIPELINE :{e}", exc_info=Tru
e)
        raise

if __name__ == '__main__':
    run_pipeline()
```

Livrable Étape 2

- Code Python complet et fonctionnel dans un dépôt Git (GitHub / GitLab)
- Fichier `README.md` expliquant comment lancer le pipeline
- Fichier `rapport_transformations.md` documentant **toutes les règles de transformation** appliquées, avec pour chaque règle :

- La règle métier
- Le code appliqué
- Le nombre de lignes affectées (issu des logs)

Étape 3 — Implémentation PostgreSQL (20%)

3.1 Création des schémas et tables

```
-- Schémas dédiés
CREATE SCHEMA IF NOT EXISTS staging_mexora;
CREATE SCHEMA IF NOT EXISTS dwh_mexora;
CREATE SCHEMA IF NOT EXISTS reporting_mexora;

-- Dimension Temps
CREATE TABLE dwh_mexora.dim_temps (
    id_date           INTEGER PRIMARY KEY,  -- format YYYYM
MDD
    jour              SMALLINT NOT NULL CHECK (jour BETWEEN
1 AND 31),
    mois              SMALLINT NOT NULL CHECK (mois BETWEEN
1 AND 12),
    trimestre         SMALLINT NOT NULL CHECK (trimestre BE
TWEEN 1 AND 4),
    annee             SMALLINT NOT NULL,
    semaine           SMALLINT,
    libelle_jour      VARCHAR(20),
    libelle_mois      VARCHAR(20),
    est_weekend       BOOLEAN DEFAULT FALSE,
    est_ferie_maroc   BOOLEAN DEFAULT FALSE,
    periode_ramadan   BOOLEAN DEFAULT FALSE
);

-- Dimension Produit (avec colonnes SCD Type 2)
CREATE TABLE dwh_mexora.dim_produit (
    id_produit_sk     SERIAL PRIMARY KEY,         -- surrog
ate key
    id_produit_nk     VARCHAR(20) NOT NULL,        -- natur
```

```sql
al key (source)
    nom_produit       VARCHAR(200) NOT NULL,
    categorie         VARCHAR(100),
    sous_categorie    VARCHAR(100),
    marque            VARCHAR(100),
    fournisseur       VARCHAR(100),
    prix_standard     DECIMAL(10,2),
    origine_pays      VARCHAR(50),
    -- SCD Type 2
    date_debut        DATE NOT NULL DEFAULT CURRENT_DATE,
    date_fin          DATE NOT NULL DEFAULT '9999-12-31',
    est_actif         BOOLEAN NOT NULL DEFAULT TRUE
);

-- Dimension Client
CREATE TABLE dwh_mexora.dim_client (
    id_client_sk      SERIAL PRIMARY KEY,
    id_client_nk      VARCHAR(20) NOT NULL,
    nom_complet       VARCHAR(200),
    tranche_age       VARCHAR(10),
    sexe              CHAR(1),
    ville             VARCHAR(100),
    region_admin      VARCHAR(100),
    segment_client    VARCHAR(20) CHECK (segment_client IN
('Gold', 'Silver', 'Bronze')),
    canal_acquisition VARCHAR(50),
    date_debut        DATE NOT NULL DEFAULT CURRENT_DATE,
    date_fin          DATE NOT NULL DEFAULT '9999-12-31',
    est_actif         BOOLEAN NOT NULL DEFAULT TRUE
);

-- Dimension Région
CREATE TABLE dwh_mexora.dim_region (
    id_region         SERIAL PRIMARY KEY,
    ville             VARCHAR(100) NOT NULL,
    province          VARCHAR(100),
    region_admin      VARCHAR(100),
    zone_geo          VARCHAR(50),
```

```sql
    pays                VARCHAR(50) DEFAULT 'Maroc'
);

-- Dimension Livreur
CREATE TABLE dwh_mexora.dim_livreur (
    id_livreur          SERIAL PRIMARY KEY,
    id_livreur_nk       VARCHAR(20),
    nom_livreur         VARCHAR(100),
    type_transport      VARCHAR(50),
    zone_couverture     VARCHAR(100)
);

-- Table de faits
CREATE TABLE dwh_mexora.fait_ventes (
    id_vente                BIGSERIAL PRIMARY KEY,
    id_date                 INTEGER     NOT NULL REFERENCES d
wh_mexora.dim_temps(id_date),
    id_produit              INTEGER     NOT NULL REFERENCES d
wh_mexora.dim_produit(id_produit_sk),
    id_client               INTEGER     NOT NULL REFERENCES d
wh_mexora.dim_client(id_client_sk),
    id_region               INTEGER     NOT NULL REFERENCES d
wh_mexora.dim_region(id_region),
    id_livreur              INTEGER     REFERENCES dwh_mexor
a.dim_livreur(id_livreur),
    -- Mesures additives
    quantite_vendue         INTEGER     NOT NULL CHECK (quant
ite_vendue > 0),
    montant_ht              DECIMAL(12,2) NOT NULL,
    montant_ttc             DECIMAL(12,2) NOT NULL,
    cout_livraison          DECIMAL(8,2),
    -- Mesures semi-additives
    delai_livraison_jours SMALLINT,
    -- Mesures non-additives (taux — à recalculer)
    remise_pct              DECIMAL(5,2) DEFAULT 0,
    -- Métadonnées ETL
    date_chargement         TIMESTAMP DEFAULT CURRENT_TIMESTA
MP,
```

```
    statut_commande       VARCHAR(20) CHECK (statut_command
e IN ('livré','annulé','en_cours','retourné'))
);
```

3.2 Indexation

```
-- Index sur les clés étrangères (jointures analytiques)
CREATE INDEX idx_fv_date     ON dwh_mexora.fait_ventes(id_d
ate);
CREATE INDEX idx_fv_produit  ON dwh_mexora.fait_ventes(id_p
roduit);
CREATE INDEX idx_fv_client   ON dwh_mexora.fait_ventes(id_c
lient);
CREATE INDEX idx_fv_region   ON dwh_mexora.fait_ventes(id_r
egion);
CREATE INDEX idx_fv_livreur  ON dwh_mexora.fait_ventes(id_l
ivreur);

-- Index composites pour les requêtes analytiques fréquente
s
CREATE INDEX idx_fv_date_region  ON dwh_mexora.fait_ventes
(id_date, id_region)
    INCLUDE (montant_ttc, quantite_vendue);
CREATE INDEX idx_fv_statut_actif ON dwh_mexora.fait_ventes
(statut_commande)
    WHERE statut_commande = 'livré';
```

3.3 Vues matérialisées (3 obligatoires)

Vue 1 — CA mensuel par région et catégorie :

```
CREATE MATERIALIZED VIEW reporting_mexora.mv_ca_mensuel AS
SELECT
    t.annee,
    t.mois,
    t.libelle_mois,
    t.periode_ramadan,
    r.region_admin,
```

```sql
    r.zone_geo,
    p.categorie,
    SUM(f.montant_ttc)           AS ca_ttc,
    SUM(f.montant_ht)            AS ca_ht,
    COUNT(DISTINCT f.id_client)  AS nb_clients_actifs,
    SUM(f.quantite_vendue)       AS volume_vendu,
    AVG(f.montant_ttc)           AS panier_moyen,
    COUNT(DISTINCT f.id_vente)   AS nb_commandes
FROM dwh_mexora.fait_ventes f
JOIN dwh_mexora.dim_temps   t ON f.id_date    = t.id_date
JOIN dwh_mexora.dim_region  r ON f.id_region  = r.id_region
JOIN dwh_mexora.dim_produit p ON f.id_produit = p.id_produi
t_sk
WHERE f.statut_commande = 'livré'
GROUP BY t.annee, t.mois, t.libelle_mois, t.periode_ramada
n,
         r.region_admin, r.zone_geo, p.categorie
WITH DATA;

CREATE INDEX ON reporting_mexora.mv_ca_mensuel(annee, moi
s);
CREATE INDEX ON reporting_mexora.mv_ca_mensuel(region_admi
n);
CREATE INDEX ON reporting_mexora.mv_ca_mensuel(categorie);
```

Vue 2 — Top produits par trimestre :

```sql
CREATE MATERIALIZED VIEW reporting_mexora.mv_top_produits A
S
SELECT
    t.annee,
    t.trimestre,
    p.nom_produit,
    p.categorie,
    p.marque,
    SUM(f.quantite_vendue)      AS qte_totale,
    SUM(f.montant_ttc)          AS ca_total,
    COUNT(DISTINCT f.id_client) AS nb_clients_distincts,
```

```
    RANK() OVER (
        PARTITION BY t.annee, t.trimestre, p.categorie
        ORDER BY SUM(f.montant_ttc) DESC
    ) AS rang_dans_categorie
FROM dwh_mexora.fait_ventes f
JOIN dwh_mexora.dim_temps   t ON f.id_date    = t.id_date
JOIN dwh_mexora.dim_produit p ON f.id_produit = p.id_produi
t_sk
WHERE f.statut_commande = 'livré'
GROUP BY t.annee, t.trimestre, p.nom_produit, p.categorie,
p.marque
WITH DATA;
```

Vue 3 — Performance livreurs (taux de retard) :

```
CREATE MATERIALIZED VIEW reporting_mexora.mv_performance_li
vreurs AS
SELECT
    l.nom_livreur,
    l.zone_couverture,
    t.annee,
    t.mois,
    COUNT(*)                                    AS nb_livra
isons,
    AVG(f.delai_livraison_jours)                AS delai_mo
yen_jours,
    COUNT(*) FILTER (WHERE f.delai_livraison_jours > 3) AS
nb_livraisons_retard,
    ROUND(
        COUNT(*) FILTER (WHERE f.delai_livraison_jours > 3)
* 100.0
        / NULLIF(COUNT(*), 0), 2
    )                                           AS taux_ret
ard_pct
FROM dwh_mexora.fait_ventes f
JOIN dwh_mexora.dim_livreur l ON f.id_livreur = l.id_livreu
r
JOIN dwh_mexora.dim_temps   t ON f.id_date    = t.id_date
```

```sql
WHERE f.statut_commande IN ('livré', 'retourné')
  AND f.delai_livraison_jours IS NOT NULL
GROUP BY l.nom_livreur, l.zone_couverture, t.annee, t.mois
WITH DATA;
```

Livrable Étape 3

- Script SQL complet de création du DWH (`create_dwh.sql`)
- Script SQL de vérification de l'intégrité référentielle (`check_integrity.sql`)
- Capture d'écran de `pgAdmin` ou `psql` montrant les tables créées et le nombre de lignes

---

Étape 4 — Dashboard de visualisation (15%)

4.1 Questions analytiques obligatoires (5 questions)

Votre dashboard doit répondre visuellement aux 5 questions suivantes :

Question 1 — Évolution du CA
> Quelle région génère le plus de CA ? Montrer l'évolution mensuelle sur 12 mois avec comparaison N-1.

- Visualisation : courbe d'évolution + carte choroplèthe du Maroc
- KPI à afficher : CA total, évolution %, top région

Question 2 — Top produits trimestriels à Tanger
> Quels sont les 10 produits les plus vendus par trimestre à Tanger ?

- Visualisation : barres horizontales triées par CA
- Filtre interactif : trimestre et année

Question 3 — Analyse par segment client
> Quel segment client (Gold/Silver/Bronze) génère le panier moyen le plus élevé ? Comment se répartissent les revenus par segment ?

- Visualisation : donut (répartition CA) + tableau (panier moyen par segment)

Question 4 — Taux de retour
> Quel est le taux de retour par catégorie de produit ? Y a-t-il des catégories problématiques ?

- Visualisation : barres horizontales avec seuil d'alerte à 5%

- Alerte rouge si taux > 5%, orange entre 3% et 5%, vert < 3%

Question 5 — Effet Ramadan
> Y a-t-il un effet Ramadan visible sur les ventes d'alimentation ? Comment se compare le volume vendu pendant vs hors Ramadan ?

- Visualisation : courbe avec zones Ramadan surlignées
- Calcul : indice de sur/sous-performance Ramadan vs moyenne

4.2 Calculs SQL pour les KPIs du dashboard

```sql
-- KPI 1 : CA du mois en cours vs mois précédent
WITH ca_mensuel AS (
    SELECT
        annee, mois,
        SUM(ca_ttc) AS ca_total
    FROM reporting_mexora.mv_ca_mensuel
    GROUP BY annee, mois
)
SELECT
    ca_total
AS ca_mois_actuel,
    LAG(ca_total) OVER (ORDER BY annee, mois)
AS ca_mois_precedent,
    ROUND(
        (ca_total - LAG(ca_total) OVER (ORDER BY annee, moi
s))
        / NULLIF(LAG(ca_total) OVER (ORDER BY annee, mois),
0) * 100,
        2
    )
AS evolution_pct
FROM ca_mensuel
ORDER BY annee DESC, mois DESC
LIMIT 2;

-- KPI 2 : Panier moyen par segment
SELECT
    c.segment_client,
```

```sql
    COUNT(DISTINCT f.id_vente)                          AS
nb_commandes,
    ROUND(SUM(f.montant_ttc) / COUNT(DISTINCT f.id_vente)::
decimal, 2) AS panier_moyen,
    ROUND(SUM(f.montant_ttc), 2)                        AS
ca_total,
    ROUND(SUM(f.montant_ttc) * 100.0
        / SUM(SUM(f.montant_ttc)) OVER (), 2)           AS
pct_ca_total
FROM dwh_mexora.fait_ventes f
JOIN dwh_mexora.dim_client c ON f.id_client = c.id_client_s
k
WHERE f.statut_commande = 'livré'
  AND c.est_actif = TRUE
GROUP BY c.segment_client
ORDER BY panier_moyen DESC;

-- KPI 3 : Taux de retour par catégorie
SELECT
    p.categorie,
    COUNT(*) FILTER (WHERE f.statut_commande = 'retourné')
AS nb_retours,
    COUNT(*)
AS nb_total,
    ROUND(
        COUNT(*) FILTER (WHERE f.statut_commande = 'retourn
é') * 100.0
        / NULLIF(COUNT(*), 0), 2
    )
AS taux_retour_pct
FROM dwh_mexora.fait_ventes f
JOIN dwh_mexora.dim_produit p ON f.id_produit = p.id_produi
t_sk
GROUP BY p.categorie
ORDER BY taux_retour_pct DESC;

-- KPI 4 : Effet Ramadan sur l'alimentation
SELECT
```

```sql
    t.periode_ramadan,
    ROUND(AVG(ca_ttc), 2)     AS ca_moyen_journalier,
    SUM(volume_vendu)         AS volume_total
FROM reporting_mexora.mv_ca_mensuel
WHERE categorie = 'Alimentation'
  AND annee >= 2022
GROUP BY t.periode_ramadan;
```

Livrable Étape 4

- Fichier Power BI (`.pbix`) ou lien Metabase partagé
- Capture d'écran du dashboard final (5 pages minimum)
- Document d'1 page expliquant les **insights clés** découverts dans les données Mexora

---

📊 Grille d'évaluation

Étape	Critères	Points
Étape 1 — Modélisation	Schéma correct et complet	5
	Granularité justifiée	5
	Additivité des mesures	5
	SCD correctement identifiés et justifiés	10
Étape 2 — ETL Python	Qualité du code (lisibilité, structure)	10
	Exhaustivité des transformations	15
	Logging et documentation des règles	10
	Gestion des erreurs	5
Étape 3 — PostgreSQL	Schéma SQL correct	5
	Indexation appropriée	5
	3 vues matérialisées fonctionnelles	10
Étape 4 — Dashboard	5 questions répondues visuellement	10
	Qualité de la visualisation	5
	Insights métier identifiés	5
TOTAL		**105**

> Les 5 points supplémentaires (sur 105) constituent des points bonus pour la qualité exceptionnelle.

🛠️ Stack technique recommandé

Outil	Usage	Installation
Python 3.11+	ETL	`brew install python` / python.org
pandas	Transformation de données	`pip install pandas`
SQLAlchemy	Connexion PostgreSQL	`pip install sqlalchemy psycopg2`
PostgreSQL 15+	Data Warehouse	postgresql.org
DBeaver	Interface SQL	dbeaver.io (gratuit)
Metabase	Dashboard (option 1)	metabase.com (gratuit, docker)
Power BI Desktop	Dashboard (option 2)	Microsoft Store (gratuit)
Git + GitHub	Versioning du code	git-scm.com

📋 Livrables finaux récapitulatifs

#	Livrable	Format
L1	Schéma entité-relation annoté	Image (PNG/PDF)
L2	Document de justification des choix	PDF (2 pages max)
L3	Code Python ETL complet	Dépôt Git
L4	Rapport des transformations	Markdown
L5	Scripts SQL création DWH	Fichier `.sql`
L6	Script SQL vérification intégrité	Fichier `.sql`
L7	Dashboard final	`.pbix` ou lien Metabase
L8	Document d'insights métier	PDF (1 page)